DATA SCIENCE / GUIA TÉCNICO EP

Fundamentos de Python

Tipos, operadores, condicionais, strings e validação de entradas

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Tipos, identidade e igualdade • 2. Operadores e fluxo de controle • 3. Strings e formatação • 4. Exceções, módulos e testes • 5. Verdade, ausência e igualdade • 6. Conversões e erros previsíveis • 7. Biblioteca padrão e caminhos • 8. Design de exercícios verificáveis

Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

01 Tipos, identidade e igualdade

Nomes em Python referenciam objetos. int, float, bool e str são tipos fundamentais; == compara valores enquanto is compara identidade. A tipagem é dinâmica, mas os tipos pertencem aos objetos e não devem ser confundidos com conversões implícitas.

Exemplo executável

```
preco = 79.90
quantidade = 3
total = preco * quantidade
assert total >= 200
assert (11 // 4, 11 % 4) == (2, 3)
```

Atenção técnica. Para dinheiro sujeito a exatidão, use decimal.Decimal; float pode introduzir diferenças binárias de representação.

Exercício de consolidação. Experimente comparar 0.1 + 0.2 com 0.3 e explicar por que a igualdade falha.

02 Operadores e fluxo de controle

if/elif/else seleciona ações a partir de condições; and/or usam curto-circuito e retornam operandos, não necessariamente bool. Evite condições dependentes de verdade implícita quando zero, None e texto vazio tiverem sentidos diferentes.

Exemplo executável

```
def classificar(nota):
    if not 0 <= nota <= 10:
        raise ValueError("Nota invalida")
    return "aprovado" if nota >= 7 else "revisar"

assert classificar(7) == "aprovado"
```

Atenção técnica. Inclua testes nas fronteiras: -1, 0, 6.99, 7, 10 e 11; também valide NaN quando aplicável.

Exercício de consolidação. Reescreva a função para rejeitar valores não numéricos e não finitos.

03 Strings e formatação

Strings são sequências imutáveis Unicode. Índices começam em zero, fatias são semiabertas e f-strings ajudam a construir mensagens. strip, casefold e split têm propósitos diferentes. Para normalização de texto, cuidado com acentos e diferenças de locale.

Exemplo executável

```
def apresentar(nome, idade):
    nome = nome.strip().title()
    return f"{nome}: {idade} anos"

assert apresentar("  ana ", 23) == "Ana: 23 anos"
assert "dados"[1:4] == "ado"
```

Atenção técnica. input() sempre retorna str; converta e trate ValueError para evitar terminar inesperadamente a execução.

Exercício de consolidação. Escreva uma função que normalize cidade e UF mantendo siglas em caixa alta.

04 Exceções, módulos e testes

Separe validação de apresentação. Use raise para erros de contrato; try/except para recuperar erros previstos; else e finally quando for útil. Imports da biblioteca padrão (math, pathlib, statistics, decimal) não precisam constar em requirements.txt.

Exemplo executável

```
from math import isfinite

def media_valida(valores):
    if not valores or not all(isfinite(x) for x in valores):
        raise ValueError("Entrada invalida")
    return sum(valores) / len(valores)

assert media_valida([6.0, 8.0]) == 7.0
```

Atenção técnica. Evite except Exception: pass; esconder falhas dificulta diagnósticos e testes.

Exercício de consolidação. Crie dois testes que verifiquem a exceção para listas vazias e valores não finitos.

05 Verdade, ausência e igualdade

Em decisões, None representa ausência explícita, enquanto 0, string vazia e lista vazia são valores falsy com semânticas diferentes. Comparações encadeadas executam cada operando no contexto da comparação. O operador is deve ser reservado principalmente para identidade e sentinelas, como is None.

Exemplo executável

```
def apresentar_resultado(valor):
    if valor is None:
        return "nao informado"
    if valor == 0:
        return "zero registrado"
    return str(valor)
assert apresentar_resultado(0) != apresentar_resultado(None)
```

Atenção técnica. Um if not valor: mistura o zero com a ausência; somente use quando isso for intencional.

Exercício de consolidação. Implemente validação que aceite zero e rejeite None.

06 Conversões e erros previsíveis

Conversões int, float, str e bool seguem regras distintas. int("3.5") falha; float("nan") é um float válido, mas não finito. Para entradas humanas, isole leitura, normalização, validação e cálculo para facilitar testes sem interação.

Exemplo executável

```
from math import isfinite
def converter_preco(texto):
    valor = float(texto.replace(",", "."))
    if not isfinite(valor) or valor < 0:
        raise ValueError("Preco invalido")
    return valor
assert converter_preco("12,50") == 12.5
```

Atenção técnica. Tratar vírgula como decimal é apenas uma convenção simplificada; separadores de milhar exigem uma regra explícita.

Exercício de consolidação. Teste "-1", "nan", "12,50" e "texto".

07 Biblioteca padrão e caminhos

A biblioteca padrão oferece recursos importantes antes de instalar bibliotecas: pathlib para caminhos, collections para contagens, statistics para resumos, decimal para valores decimais e datetime para datas. Caminhos relativos dependem do diretório atual do processo.

Exemplo executável

```
from pathlib import Path
from collections import Counter
frequencias = Counter("banana")
assert frequencias["a"] == 3
caminho = Path("outputs") / "exemplo.csv"
print(caminho.parent)
```

Atenção técnica. Não presuma que Path.cwd() seja a pasta do notebook em todos os ambientes; confirme antes de ler arquivos.

Exercício de consolidação. Crie um caminho relativo que funcione no Linux e Windows.

08 Design de exercícios verificáveis

Um exemplo didático útil apresenta entrada, transformação, resultado e condição de aceitação. Assertions servem para verificações locais, mas podem ser desabilitadas em execução otimizada; validações de negócio devem usar exceções explícitas.

Exemplo executável

```
def dividir(a, b):
    if b == 0:
        raise ZeroDivisionError("Divisor igual a zero")
    return a / b
assert dividir(9, 3) == 3
assert isinstance(dividir(1, 2), float)
```

Atenção técnica. Não use assert para rejeitar entradas inválidas em sistemas de produção; preferir if e raise.

Exercício de consolidação. Construa três testes positivos e dois negativos para dividir.

Checklist de revisão

Verifique	Evidência
Conceito	Consegue explicar sem copiar o código?
Execução	O exemplo produz o resultado esperado?
Casos-limite	Entradas inválidas e extremas foram testadas?
Reprodutibilidade	O kernel e as dependências estão registrados?

Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.